

Zadanie 2

Otwarto: poniedziałek, 20 kwietnia 2026, 00:00

Wymagane do: niedziela, 24 maja 2026, 23:59

1. Klient radia internetowego

Zaimplementuj klienta radia internetowego. Klient komunikuje się z serwerem za pomocą TCP i IPv4 lub IPv6. Rozpracowanie szczegółów protokołu komunikacyjnego jest częścią zadania. Proszę przeanalizować załączone na końcu przykłady.

2. Parametry uruchamiania klienta

Klienta uruchamia się poleceniem `sikradio` z następującymi parametrami:

- `-u url` – napis identyfikujący serwer i udostępniany przez niego strumień audio, parametr obowiązkowy;
- `-m` – wysłanie do serwera prośby o multipleksowanie ze strumieniem audio informacji tekstowych, parametr opcjonalny;
- `-t timeout` – limit czasu, po którym, jeśli nie nadeszły kolejne dane, klient rozłącza się i ponawia połączenie z serwerem, wartość w milisekundach z przedziału od 100 do 100000, parametr opcjonalny, domyślnie 5000;
- `-4` – wymuszenie użycia IPv4, parametr opcjonalny;
- `-6` – wymuszenie użycia IPv6, parametr opcjonalny;
- `-v verbosity` – zakres informacji wypisywanych na standardowe wyjście diagnostyczne, wartość `verbosity` jest liczbą całkowitą od 0 do 4, parametr opcjonalny, domyślnie 2;
- `-q` – skrót dla parametru `-v0`.

Jeśli podano oba parametry `-4` i `-6` lub nie podano żadnego z nich, klient powinien wybrać wersję IP wynikającą z pierwszego przypisanego serwerowi adresu IP (patrz funkcja `getaddrinfo`).

Parametry mogą być podawane w dowolnej kolejności. Między nazwą parametru a jego wartością nie musi być odstęp. Parametry mogą być zblokowane, np. `-m46` oznacza podanie parametrów `m`, `-4` i `-6`. Zachowanie programu, gdy któryś z parametrów został podany wielokrotnie, powinno być rozsądne.

3. Wypisywane informacje

Klient nie interpretuje zawartości odbieranego strumienia audio i wypisuje go bez zmian na standardowe wyjście. Do dekodowania i odtwarzania strumienia audio używamy zewnętrznego programu. Przykłady użycia klienta i programu `play` są zamieszczone w załączonych poniżej plikach. Alternatywą dla tego programu jest program `mpv`. Wystarczy zamienić wywołanie `play -q -t mp3 -` na `mpv --really-quiet -`.

Ewentualne dane tekstowe multipleksowane ze strumieniem audio klient wypisuje bez zmian na standardowe wyjście diagnostyczne.

Informacje o błędnym wywołaniu (błędnych parametrach) klient wypisuje na standardowe wyjście diagnostyczne.

Oprócz tego klient wypisuje na standardowe wyjście diagnostyczne dodatkowe informacje wymienione w poniższych punktach, których numer nie przekracza podanej wartości `verbosity`:

- 0 – nie wypisuje żadnych dodatkowych informacji;
- 1 – wypisuje informacje o przebiegu komunikacji z serwerem;
- 2 – wypisuje komunikaty o błędach krytycznych uniemożliwiających kontynuowanie pracy, a zwykle związanych z wywołaniem funkcji systemowych lub bibliotecznych;
- 3 – wypisuje komunikaty o błędach niekrytycznych nieuniemożliwiających kontynuowania pracy, a zwykle związanych z wywołaniem funkcji systemowych lub bibliotecznych;
- 4 – wypisuje komunikaty diagnostyczne ułatwiające debugowanie.

4. Kończenie programu

Jeśli użytkownik wpisał `quit` i nacisnął `Enter`, klient zamyka połączenie z serwerem, wypisuje wszystkie dotychczas odebrane dane i kończy się statusem 0.

Jeśli serwer zamknął połączenie, klient wypisuje wszystkie dotychczas odebrane dane i kończy się statusem 0.

Jeśli klient został błędnie wywołany (z błędnymi parametrami), kończy się statusem 1.

Jeśli wystąpił błąd krytyczny uniemożliwiający kontynuowanie pracy, klient kończy się statusem 1.

5. Rozwiązanie

Rozwiązanie należy zaimplementować w języku C lub C++, korzystając z interfejsu gniazd. Sugerujemy skorzystanie z bibliotek `libssl` i `libcrypto`. Nie wolno korzystać z żadnych innych bibliotek realizujących komunikację sieciową. Program ma się kompilować i uruchamiać w laboratorium komputerowym zarówno na maszynie `students`, jak i na maszynach w labach.

Program powinien być napisany zgodnie ze sztuką. Brak oczywistych oczekiwań wobec programu (np. że nie formatują dysku, czy że wartości zwracane przez funkcje systemowe są sprawdzane) w treści zadania nie oznacza, że program, który ich nie spełnia, będzie uznany za dobry. Również kod programu będzie podlegał ocenie. Testy będą rygorystycznie sprawdzać poprawność odbieranego strumienia audio i multipleksowanych z nim informacji tekstowych. Strumień audio powinien być odtwarzany bez zacięć i zniekształceń.

Jako rozwiązanie należy przysłać archiwum zawierające pliki niezbędne do zbudowania rozwiązania. Nie wolno załączać plików binarnych i innych zbędnych. Do stworzenia archiwum należy użyć programu `zip`, `rar`, `7z` lub pary programów `tar` i `gzip`. Archiwum powinno mieć odpowiednio rozszerzenie `.zip`, `.rar`, `.7z` lub `.tgz`. Po rozpakowaniu wszystkie pliki powinny znajdować się w katalogu, w którym jest plik archiwum. Archiwum nie może zawierać podkatalogów. Archiwum powinno zawierać plik `makefile` lub `Makefile`. Wykonanie polecenia `make` powinno stworzyć plik wykonywalny `sikradio`. Wykonanie polecenia `make clean` powinno usunąć wszystkie pliki powstałe podczas kompilowania.

6. Przykłady użycia

sikradio_example_1.log	16 lutego 2026, 16:51
sikradio_example_2.log	16 lutego 2026, 16:51
sikradio_example_3.log	16 lutego 2026, 16:51
sikradio_example_4.log	16 lutego 2026, 16:51
sikradio_example_5.log	16 lutego 2026, 16:51
sikradio_example_6.log	30 kwietnia 2026, 13:46
sikradio_example_7.log	30 kwietnia 2026, 13:46